



UNIVERSIDAD AUTONOMA DEL BENI "JOSE BALLIVIAN"

FACULTAD DE INGENIERIA Y TECNOLOGIA

CAPIVERSE - SISTEMA DE REGISTRO DE TAREAS COLABORATIVAS EN TIEMPO REAL

ASIGNATURA: Tecnologías Emergentes

DOCENTE: Ing. Hermes Rodríguez Rivero

ESTUDIANTES:

- Humberto Orellano justiniano
- Guido Diogo Alaiza Gómez
- Luis Andrés Mariscal Farell

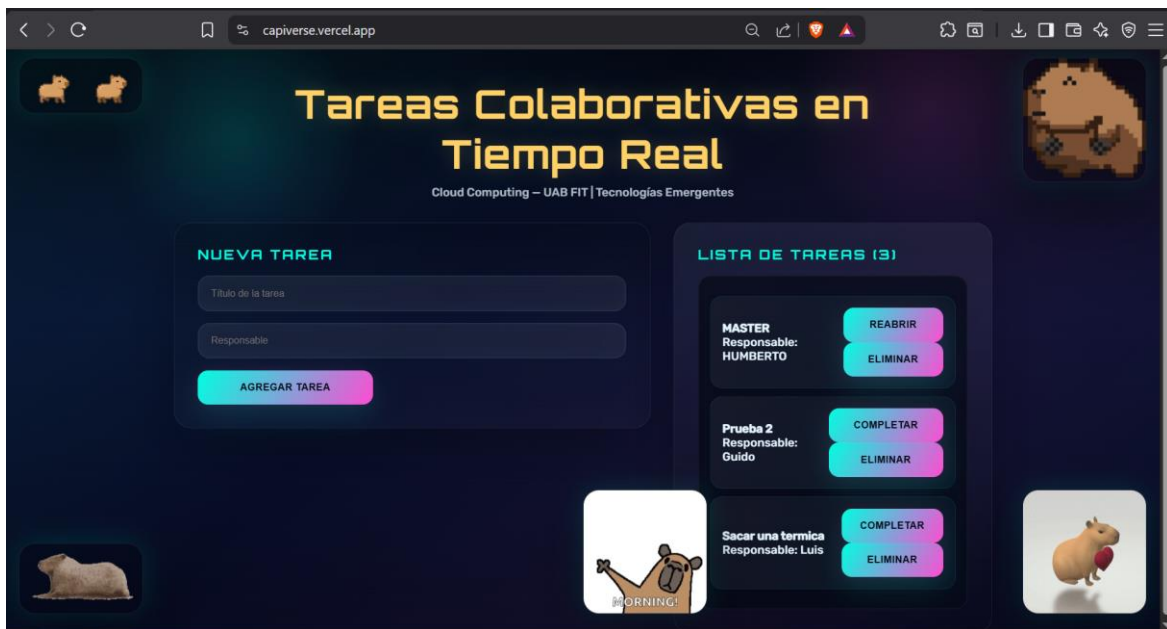
FECHA: 18/06/2026

INDICE

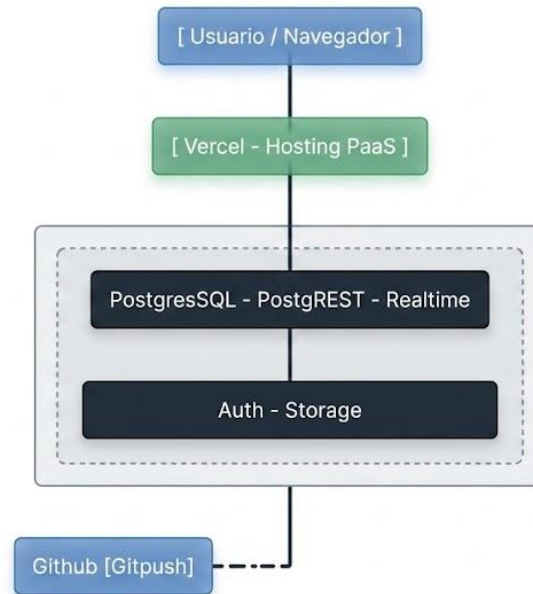
1. INTRODUCCIÓN Y DESCRIPCIÓN DEL PROYECTO	3
2. DESCRIPCIÓN DE LA ARQUITECTURA CLOUD IMPLEMENTADA	4
Desglose de Componentes:	4
3. CLASIFICACIÓN DEL MODELO DE SERVICIO (IaaS / PaaS / SaaS).....	5
4. RESPUESTAS A LAS PREGUNTAS GUÍA (REFLEXIÓN TÉCNICA)	6
1. ¿En qué modelo de servicio clasificarías a Supabase y por qué?.....	6
2. ¿Qué tareas del backend manejó Supabase que normalmente tendrías que hacer manualmente?	6
3. ¿Cómo funciona el tiempo real en Supabase? ¿Qué tecnología subyacente utiliza?	7
4. ¿Cuál fue el proceso de CI/CD que implementaste con GitHub y Vercel?	8
5. ¿Qué ventajas y desventajas encontraste en el modelo PaaS comparado con un servidor tradicional?	9
6. ¿Qué consideraciones de seguridad deberías aplicar si esta fuera una aplicación en producción real?	10
5. REFLEXIÓN GRUPAL Y CONCLUSIONES.....	10

1. INTRODUCCIÓN Y DESCRIPCIÓN DEL PROYECTO

Capiverse es una aplicación web desarrollada con el propósito fundamental de registrar y mostrar tareas en tiempo real utilizando herramientas 100% gratuitas y de alta demanda en el mercado global. El sistema está diseñado bajo un enfoque colaborativo, permitiendo que múltiples usuarios interactúen simultáneamente creando, completando o eliminando tareas sin necesidad de recargar la interfaz gráfica, lo que optimiza significativamente la experiencia del usuario y demuestra de forma práctica las capacidades del Cloud Computing y los modelos Serverless.



2. DESCRIPCIÓN DE LA ARQUITECTURA CLOUD IMPLEMENTADA



Desglose de Componentes:

- **Capa de Cliente (Usuario / Navegador):** Representa el frontend estático que interactúa de cara al usuario. Ejecuta la estructura lógica y los estilos que consumen de forma directa el SDK de Supabase a través de peticiones HTTPS y túneles persistentes de comunicación.
- **Capa de Hosting PaaS (Vercel):** Se encarga de alojar la aplicación y servir los recursos estáticos globalmente mediante una CDN. Vercel abstrae por completo la necesidad de aprovisionar o configurar servidores web.
- **Capa de Backend PaaS/BaaS (Supabase):** Es el motor centralizado:
 - **PostgreSQL:** Base de datos relacional nativa en la nube encargada del almacenamiento seguro y persistente de las tareas.
 - **PostgREST:** Mapeador automático que expone la base de datos de manera directa a través de una API RESTful limpia.
 - **Realtime:** Módulo basado en WebSockets encargado de retransmitir los cambios de la base de datos de forma inmediata a todos los clientes.
 - **Auth y Storage:** Módulos integrados listos para su escalabilidad futura en el control de usuarios y archivos.
- **Capa de Integración (GitHub):** El repositorio remoto que actúa como el disparador del flujo de automatización (CI/CD) mediante operaciones de git push.

3. CLASIFICACIÓN DEL MODELO DE SERVICIO (IaaS / PaaS / SaaS)

Basado en las tecnologías aplicadas y listadas en, el ecosistema de herramientas se clasifica bajo los siguientes modelos de computación en la nube:

#	Herramienta	Modelo de Servicio	Justificación Técnica
1	Visual Studio Code	Entorno Local / ID	Herramienta de escritorio local utilizada como el entorno de desarrollo integrado principal para la codificación.
2	Node.js 20 LTS	Entorno de Ejecución	Proporciona el ecosistema de ejecución para gestionar herramientas locales de desarrollo.
3	Git	Control de Versiones Local	Administra el historial de cambios del código fuente localmente en la máquina del desarrollador.
4	GitHub	SaaS (Software as a Service)	Plataforma completamente administrada en la nube que consumimos como software para almacenar repositorios remotos y gestionar el trabajo colaborativo.
5	Supabase	PaaS (Platform as a Service / BaaS)	Entrega una plataforma de infraestructura preconfigurada (Base de datos, APIs y WebSockets) lista para el desarrollo, sin gestión de sistemas operativos.
6	Vercel	PaaS (Platform as a Service)	Proporciona la plataforma completa para compilar, alojar y distribuir aplicaciones web de forma automatizada mediante flujos CI/CD.

4. RESPUESTAS A LAS PREGUNTAS GUÍA (REFLEXIÓN TÉCNICA)

1. ¿En qué modelo de servicio clasificarías a Supabase y por qué?

Se clasifica inequívocamente como **PaaS** (Plataforma como Servicio), adoptando la variante específica de **BaaS** (Backend as a Service). Esto se debe a que el proveedor de la nube nos entrega un entorno de desarrollo completamente operativo con abstracción absoluta de la infraestructura física. El equipo no tuvo que preocuparse por la instalación del motor de bases de datos, asignación de memoria RAM al servidor, ni direccionamiento de redes IP; solo nos enfocamos en interactuar con la plataforma mediante código.

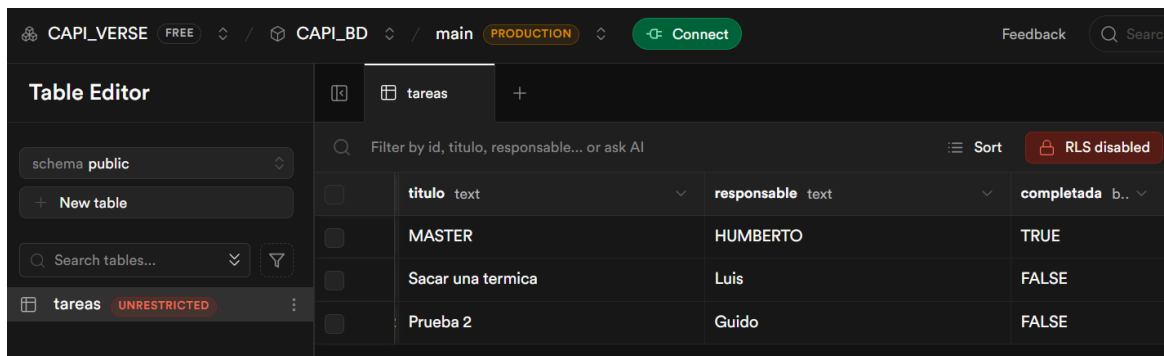
2. ¿Qué tareas del backend manejó Supabase que normalmente tendrías que hacer manualmente?

Normalmente, en un entorno de desarrollo tradicional, habríamos tenido que:

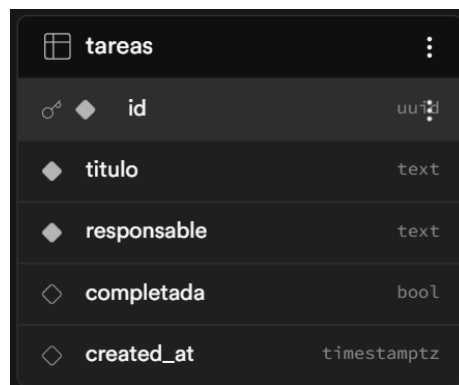
1. Instalar y configurar manualmente un sistema operativo de servidor y el software de PostgreSQL de forma local o VPS.
2. Desarrollar un servidor de aplicaciones completo (por ejemplo, con Node.js y Express) para crear manualmente las rutas API (endpoints) de inserción, actualización, eliminación y consulta de registros.
3. Programar la capa de conexión hacia la base de datos (ORMs o queries nativas) protegiendo el sistema de inyecciones SQL.
4. Levantar, configurar y mantener un servidor de WebSockets dedicado para habilitar el tiempo real, gestionando las conexiones concurrentes de los clientes.

3. ¿Cómo funciona el tiempo real en Supabase? ¿Qué tecnología subyacente utiliza?

El tiempo real en Supabase funciona escuchando de manera constante el **Write-Ahead Log (WAL)** de PostgreSQL. Cada vez que ocurre un evento transaccional (INSERT, UPDATE o DELETE) en la tabla especificada, Supabase captura ese cambio a nivel de base de datos. La tecnología subyacente utilizada para propagar el evento hacia los navegadores cliente de forma síncrona y bidireccional es **WebSockets**, transmitiendo los datos mediante canales abiertos persistentes sin sobrecargar el servidor con peticiones HTTP cíclicas.



id	titulo	responsable	completada
----	--------	-------------	------------

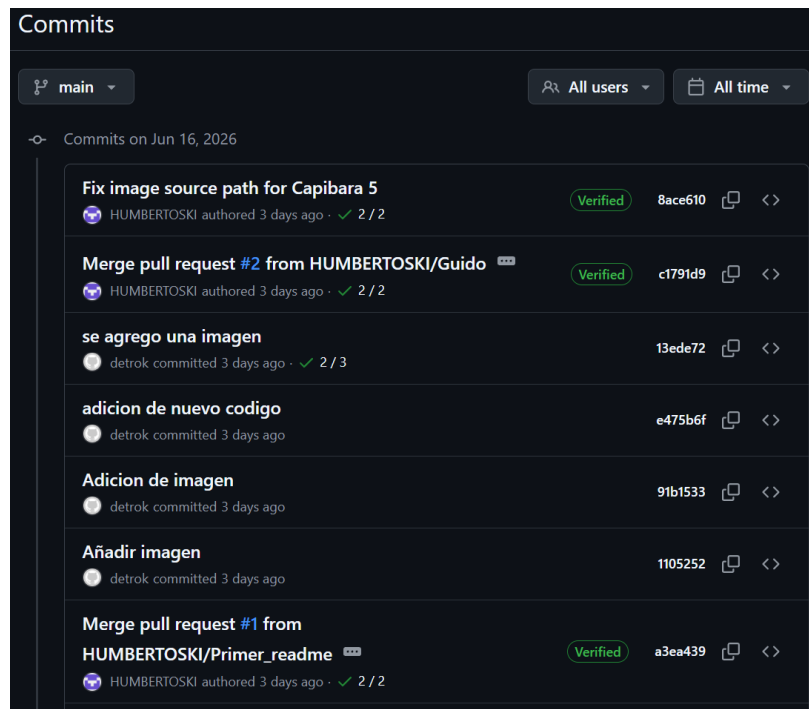


id	titulo	responsable	completada	created_at
----	--------	-------------	------------	------------

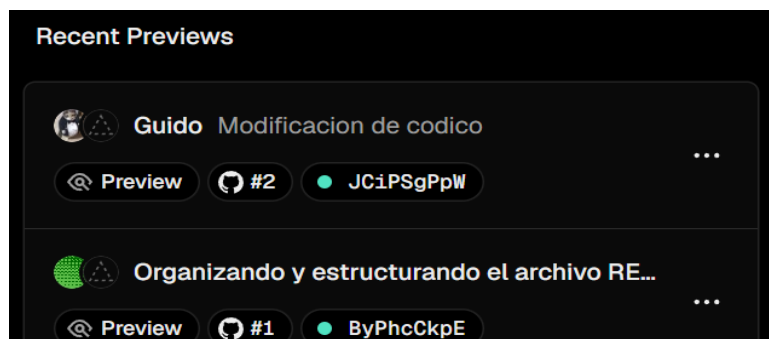
4. ¿Cuál fue el proceso de CI/CD que implementaste con GitHub y Vercel?

Se implementó un pipeline automatizado de Integración Continua (CI) y Despliegue Continuo (CD):

- **Integración Continua (CI):** Al realizar cambios en el entorno local (VS Code), los archivos modificados se consolidan mediante *commits* y se suben remotamente con un git push al repositorio público de GitHub, asegurando la consistencia del código del equipo.



- **Despliegue Continuo (CD):** Vercel se encuentra vinculado de manera nativa al repositorio de GitHub. Al detectar un cambio en la rama principal (main), activa un "Webhook" interno que descarga automáticamente el código más reciente, realiza el proceso de empaquetado y lo despliega a producción en su red global de servidores, todo en cuestión de segundos y de manera automatizada.



5. ¿Qué ventajas y desventajas encontraste en el modelo PaaS comparado con un servidor tradicional?

- **Ventajas:** Reducción drástica del *Time-to-Market* (tiempo de salida a producción), aprovisionamiento instantáneo de recursos, costo financiero cero gracias a las capas gratuitas, escalabilidad elástica automatizada y nula inversión de tiempo en mantenimiento de servidores o parches de seguridad.
- **Desventajas:** *Vendor Lock-in* (dependencia crítica del proveedor y sus tecnologías), limitaciones de configuración fina a bajo nivel en el sistema operativo o motor de base de datos, y restricciones estrictas de cuotas operativas en los planes gratuitos si el tráfico de la aplicación incrementa exponencialmente.

6. ¿Qué consideraciones de seguridad deberías aplicar si esta fuera una aplicación en producción real?

Si **Capiverse** migrara a un entorno de producción masivo, se deberían aplicar las siguientes directrices obligatorias:

1. **Restringir las Políticas RLS:** Eliminar la política permisiva de acceso académico (ALL USING (true)) que permite a cualquier usuario anónimo alterar los datos, reemplazándola por políticas basadas en usuarios validados mediante el módulo de *Supabase Auth*.
2. **Ocultar Credenciales del Lado del Cliente:** Evitar la exposición de las llaves públicas API en código duro dentro de app.js. En su lugar, es recomendable utilizar variables de entorno seguras manejadas en Vercel o pasar las peticiones críticas a través de funciones Serverless intermedias (Edge Functions).
3. **Validación Estricta de Esquemas:** Implementar restricciones y disparadores (Triggers) a nivel de base de datos para asegurar la sanitización absoluta de las entradas y prevenir la inyección de código malicioso o datos corruptos.

5. REFLEXIÓN GRUPAL Y CONCLUSIONES

La ejecución del proyecto **Capiverse** nos permitió asimilar de forma práctica cómo los paradigmas modernos de computación en la nube (PaaS y Serverless) democratizan el desarrollo de software de alta calidad. Al remover las barreras operativas y de infraestructura que tradicionalmente consumían semanas de configuración, el ingeniero de sistemas contemporáneo puede enfocar sus recursos cognitivos puramente en resolver la lógica de negocio y garantizar una interfaz de usuario óptima. La sinergia perfecta conseguida entre Git, GitHub, Supabase y Vercel demuestra que es completamente factible lanzar soluciones globales funcionales, en tiempo real y escalables utilizando metodologías ágiles y herramientas accesibles de nivel empresarial.